Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

Materia: Fundamentación de robótica

Clave: TE3001B

Grupo: 101

Profesores:
Alfredo García Suárez
Rigoberto Cerino Jiménez
Juan Manuel Ahuactzin Larios

Actividad 2.2 PWM con ESP32

Alumnos
Omar Perez Dominguez | A01738306
Ezequiel Luna Trejo | A01738153
Antonio Méndez Rodríguez | A01738269

3 de Marzo De 2026

Resumen

En el presente trabajo se desarrolló un sistema de control de velocidad para un motor DC empleando un ESP32 e integrando comunicación distribuida mediante micro-ROS y ROS 2. El sistema fue capaz de recibir comandos de dirección desde un nodo externo y publicar variables de estado, específicamente el voltaje calculado y el duty cycle del PWM, permitiendo su monitoreo en tiempo real. A través de pruebas en terminal y la visualización gráfica con rqt_plot, se validó la correcta transmisión de datos y la relación proporcional entre la entrada analógica del potenciómetro y la señal PWM generada. Asimismo, se identificó y corrigió un error en el escalamiento del ADC que ocasionaba limitaciones en el rango del PWM, logrando finalmente un funcionamiento estable y acorde con los objetivos planteados.

Objetivos

Objetivo General

Migrar un sistema de control de velocidad de un motor de corriente directa (DC), originalmente implementado de manera nativa en un ESP32 mediante modulación por ancho de pulso (PWM), hacia una arquitectura distribuida basada en micro-ROS, permitiendo su integración dentro del ecosistema de ROS 2, e implementando comunicación mediante publishers y subscribers, una máquina de estados con reconexión automática y manejo adecuado de memoria dinámica en mensajes tipo String.

Objetivos específicos

- Analizar el funcionamiento del sistema original de control de velocidad por PWM en el ESP32, identificando las variables críticas: duty cycle, valor PWM, voltaje ADC y estado del motor.
- Explicar la resolución del ADC del ESP32 y corregir el error que generaba una señal PWM incorrecta observada en el osciloscopio.
- Implementar un nodo embebido en el ESP32 usando micro-ROS para integrarlo al entorno de ROS 2.
- Configurar publishers para /duty_cycle, /voltaje y /motor_state, y un subscriber para /motor_command.

- Validar el comportamiento del sistema utilizando rqt_plot, comparando la variación del potenciómetro con la respuesta publicada.

Introducción

En los sistemas embebidos aplicados a robótica y automatización, el control de motores de corriente directa (DC) es una de las aplicaciones más comunes, ya que permite regular velocidad y dirección de movimiento en múltiples dispositivos. Tradicionalmente, este control se implementa de forma local en un microcontrolador, donde las señales de entrada son procesadas y convertidas en señales de salida que accionan el actuador. Sin embargo, en aplicaciones robóticas modernas se busca integrar estos sistemas a arquitecturas distribuidas que permitan monitoreo, control remoto y escalabilidad.

Funcionamiento del PWM

Una técnica fundamental para el control de velocidad de motores DC es la Modulación por Ancho de Pulso (PWM, Pulse Width Modulation). El PWM consiste en una señal digital periódica cuya característica principal es el ciclo de trabajo (duty cycle), que representa el porcentaje de tiempo que la señal permanece en nivel alto dentro de un periodo. Al variar el duty cycle, se modifica el valor promedio de voltaje entregado al motor, permitiendo regular su velocidad sin necesidad de variar físicamente la fuente de alimentación. En el caso del ESP32, el PWM se genera mediante canales configurables donde se define la frecuencia y la resolución en bits, lo que determina el rango de valores posibles (por ejemplo, 8 bits equivalen a valores entre 0 y 255).

Funcionamiento del Puente H

Para controlar la dirección de giro del motor se utiliza un puente H, el cual es un arreglo electrónico de transistores que permite invertir la polaridad aplicada al motor. Dependiendo de la combinación de señales de control, el motor puede girar hacia la derecha, hacia la izquierda o permanecer detenido. Este circuito es esencial cuando se requiere control bidireccional, ya que el microcontrolador por sí solo no puede suministrar la corriente necesaria para el accionamiento directo del motor.

Funcionamiento de Micro-ROS:

Con el objetivo de integrar este sistema a un entorno robótico más amplio, se emplea micro-ROS, una extensión de ROS 2 diseñada para dispositivos con recursos limitados como microcontroladores. Micro-ROS permite que el ESP32 funcione como un nodo dentro de una red ROS 2, publicando y suscribiéndose a tópicos mediante un agente que se ejecuta en una computadora anfitriona. Internamente, utiliza el protocolo DDS-XRCE, optimizado para entornos con restricciones de memoria y procesamiento.

Dentro de esta arquitectura distribuida, el sistema no solo genera la señal PWM para el motor, sino que también publica variables críticas como el duty cycle, el voltaje leído y el estado del motor, además de suscribirse a comandos externos para controlar la dirección y encendido. Para garantizar robustez en la comunicación, se implementa una máquina de estados que permite la reconexión automática en caso de pérdida de enlace con el agente, mejorando la tolerancia a fallos del sistema.

Solución del problema

Para el desarrollo de la práctica se siguió una metodología estructurada que incluyó el análisis del sistema original de control, la identificación y corrección de errores en la adquisición de datos, y finalmente la migración del sistema hacia una arquitectura distribuida basada en micro-ROS integrada al ecosistema de ROS 2.

Implementación del sistema inicial

Inicialmente, se trabajó con un código proporcionado por el profesor que iba directamente implementado en el ESP32. Este sistema leía la señal analógica proveniente de un potenciómetro mediante el convertidor analógico-digital (ADC), convertía dicha lectura en un valor de voltaje, calculaba el ciclo de trabajo (duty cycle) correspondiente y generaba una señal PWM para controlar la velocidad del motor de corriente directa. La dirección de giro se gestionaba mediante un puente H, controlado por dos pines digitales que determinaban el sentido de rotación o la detención del motor. Además, los valores calculados eran mostrados en el monitor serial para su verificación.

Error en el código y corrección

Durante la fase de validación con osciloscopio, se detectó un comportamiento no esperado en la señal PWM al variar el potenciómetro, ya que no permitía aumentar el PWM a más del 25%. Tras analizar el código, se identificó que el problema se encontraba en el mapeo del valor leído por el ADC. El código original utilizaba un rango de 0 a 1023 para convertir la lectura analógica a un valor PWM entre 0 y 255. Sin embargo, el ESP32 cuenta con un ADC de 12 bits, cuyo rango real es de 0 a 4095. El uso incorrecto del límite superior provocaba que el mapeo se saturara antes de alcanzar el valor máximo real, generando una relación incorrecta entre la posición del potenciómetro y el duty cycle aplicado. Como consecuencia, la señal observada en el osciloscopio no correspondía con la entrada analógica.

La corrección consistió en modificar el rango del mapeo a 0–4095, de modo que coincidiera con la resolución real del ADC. Esta modificación permitió restablecer la proporcionalidad entre la lectura analógica y el valor PWM generado, obteniéndose una señal lineal y coherente con la variación del potenciómetro. La justificación de esta corrección está dada porque el proceso de cuantización del ADC depende directamente de su número de bits, por lo que cualquier cálculo posterior debe considerar su resolución real para evitar errores de escalamiento.

Migración a Micro-ROS

Una vez validado el funcionamiento local del control del motor, se procedió a migrar el sistema hacia una arquitectura distribuida utilizando micro-ROS. El primer paso consistió en incluir las librerías necesarias y declarar las entidades principales del sistema, tales como el nodo, el publicador, el suscriptor y el ejecutor. Estas estructuras permiten establecer la comunicación entre el ESP32 y el agente micro-ROS que se ejecuta en la computadora.

```
#include <micro_ros_arduino.h>

#include <stdio.h>
#include <string.h>

#include <rcl/rcl.h>
#include <rcl/error_handling.h>
#include <rclc/rclc.h>
#include <rclc/executor.h>

#include <std_msgs/msg/string.h>
#include <std_msgs/msg/float32_multi_array.h>
```

Posteriormente, se definieron los pines asociados al control del motor y los parámetros del PWM, incluyendo la frecuencia y la resolución. También se declararon las variables que almacenan el estado del sistema, como la lectura del potenciómetro, el valor de PWM, el voltaje calculado y el duty cycle. Estas variables serían posteriormente publicadas para su monitoreo en ROS 2.

```
#define variador 34
#define PWM_PIN 26
#define In1 14
#define In2 27

#define freq 5000
#define resolution 8
```

El siguiente paso fue la implementación del callback del suscriptor. Esta función se ejecuta automáticamente cada vez que se recibe un mensaje en el tópico motor_cmd. Dentro de ella se interpretan los comandos recibidos: "D" para giro en una dirección, "I" para la dirección contraria y "S" para detener el motor. Dependiendo del comando, se activan los pines correspondientes del puente H y se actualiza la variable motorEnMovimiento, la cual determina si el sistema debe generar PWM o permanecer detenido. Se incluyó además una validación para evitar errores en caso de recibir mensajes nulos o vacíos.

```
//Callback
void subscription_callback(const void * msgin)
{
  const std_msgs__msg__String * msg = (const std_msgs__msg__String *)msgin;

  // Validación de seguridad por si llega un mensaje vacío
  if (msg == NULL || msg->data.data == NULL) return;

  if (strcmp(msg->data.data, "D") == 0) {
    digitalWrite(In1, HIGH);
    digitalWrite(In2, LOW);
    motorEnMovimiento = true;
  }
  else if (strcmp(msg->data.data, "I") == 0) {
    digitalWrite(In1, LOW);
    digitalWrite(In2, HIGH);
    motorEnMovimiento = true;
  }
  else if (strcmp(msg->data.data, "S") == 0) {
    digitalWrite(In1, LOW);
    digitalWrite(In2, LOW);
    ledcWrite(PWM_PIN, 0);
    motorEnMovimiento = false;
  }
}
```

En la función setup() se realizó la configuración principal del sistema. Primero se estableció el transporte serial mediante set_microros_transports(), lo que permite la comunicación con el agente micro-ROS. Luego se configuraron los pines como entradas o salidas y se inicializó el canal PWM con la frecuencia y resolución definidas previamente.

Posteriormente, se inicializaron las estructuras de soporte de micro-ROS, se creó el nodo embebido y se configuraron tanto el suscriptor al tópico motor_cmd como el publicador del tópico motor_state. El ejecutor fue configurado para gestionar la suscripción y ejecutar el callback cuando se reciben nuevos datos.

```
void setup()
{
  // Transporte serial micro-ROS
  set_microros_transports();

  pinMode(variador, INPUT);
  pinMode(In1, OUTPUT);
  pinMode(In2, OUTPUT);

  ledcAttach(PWM_PIN, freq, resolution);

  delay(2000);

  allocator = rcl_get_default_allocator();

  rclc_support_init(&support, 0, NULL, &allocator);
  rclc_node_init_default(&node, "esp32_motor_node", "", &support);

  //Subscriber
  rclc_subscription_init_default(
    &subscriber,
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, String),
    "motor_cmd");

  // Publisher
  rclc_publisher_init_default(
    &publisher,
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32MultiArray),
    "motor_state");

  rclc_executor_init(&executor, &support.context, 1, &allocator);
  rclc_executor_add_subscription(
```

Un aspecto crítico en esta etapa fue la inicialización manual de la memoria para los mensajes, especialmente para el tipo String. En micro-ROS es necesario reservar memoria dinámicamente antes de utilizar estos mensajes, por lo que se empleó malloc para definir la capacidad del buffer. De igual forma, se reservó memoria para el arreglo de tipo Float32MultiArray que almacena el voltaje y el duty cycle. Esta inicialización evita errores de serialización y posibles reinicios del microcontrolador.

```
    &executor,
    &subscriber,
    &msg_cmd,
    &subscription_callback,
    ON_NEW_DATA);
  msg_cmd.data.data = (char *) malloc(20 * sizeof(char));
  msg_cmd.data.capacity = 20;
  msg_cmd.data.size = 0;

  //Inicializar mensaje publicador
  msg_state.data.data = (float*) malloc(2 * sizeof(float));
  msg_state.data.size = 2;
  msg_state.data.capacity = 2;

  msg_state.layout.dim.capacity = 0;
  msg_state.layout.dim.size = 0;
  msg_state.layout.dim.data = NULL;
}
```

Finalmente, en la función loop() se implementó la lógica principal del sistema. En cada iteración se mantiene activo el ejecutor mediante rclc_executor_spin_some(), lo que permite procesar nuevos mensajes sin bloquear la ejecución. Si la variable motorEnMovimiento es verdadera, el sistema lee el valor del potenciómetro mediante analogRead, lo mapea correctamente del rango 0–4095 a 0–255 para generar el PWM, y actualiza la salida con ledcWrite

Además, se calculan el voltaje real y el duty cycle en porcentaje, los cuales se almacenan en el arreglo del mensaje motor_state y se publican cada 100 milisegundos. Este control de temporización evita saturar la comunicación serial y mantiene una tasa de actualización estable de aproximadamente 10 Hz.

```
void loop()
{
  rclc_executor_spin_some(&executor, RCL_MS_TO_NS(10));

  if (motorEnMovimiento)
  {
    // Solo publicar si ha pasado el tiempo establecido
    if (millis() - last_time >= publish_interval) {
      last_time = millis();

      pot = analogRead(variador);

      pwm = map(pot, 0, 4095, 0, 255);
      ledcWrite(PWM_PIN, pwm);

      voltaje = pot * (Vcc / 4095.0);
      duty = (voltaje / Vcc) * 100.0;

      msg_state.data.data[0] = voltaje;
      msg_state.data.data[1] = duty;

      rcl_publish(&publisher, &msg_state, NULL);
    }
  }
}
```

Con esta estructura, el sistema logra recibir comandos desde ROS 2, controlar la dirección y velocidad del motor mediante PWM y publicar continuamente variables de estado para su monitoreo gráfico. De esta manera, se consolidó la transición de un sistema de control local hacia una arquitectura distribuida robusta y funcional.

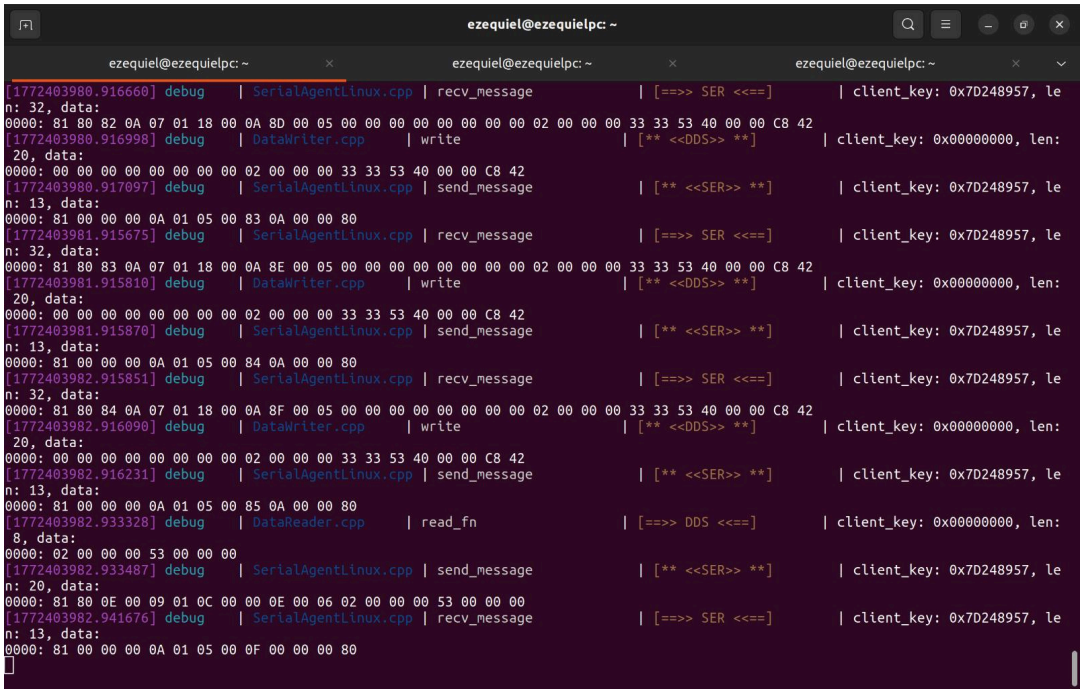
Resultados

Durante la validación del sistema migrado a una arquitectura basada en micro-ROS dentro del entorno de ROS 2, se obtuvieron diversas evidencias que permiten confirmar el cumplimiento de los objetivos planteados.

1. Comunicación con micro-ROS

Se realizó la ejecución del agente micro-ROS en la computadora anfitriona. En la terminal se observaron mensajes de depuración asociados a SerialAgentLinux.cpp, DataWriter.cpp y

DataReader.cpp, lo que confirma que el ESP32 estableció correctamente la comunicación serial con el agente y que se estaban transmitiendo y recibiendo paquetes DDS-XRCE.

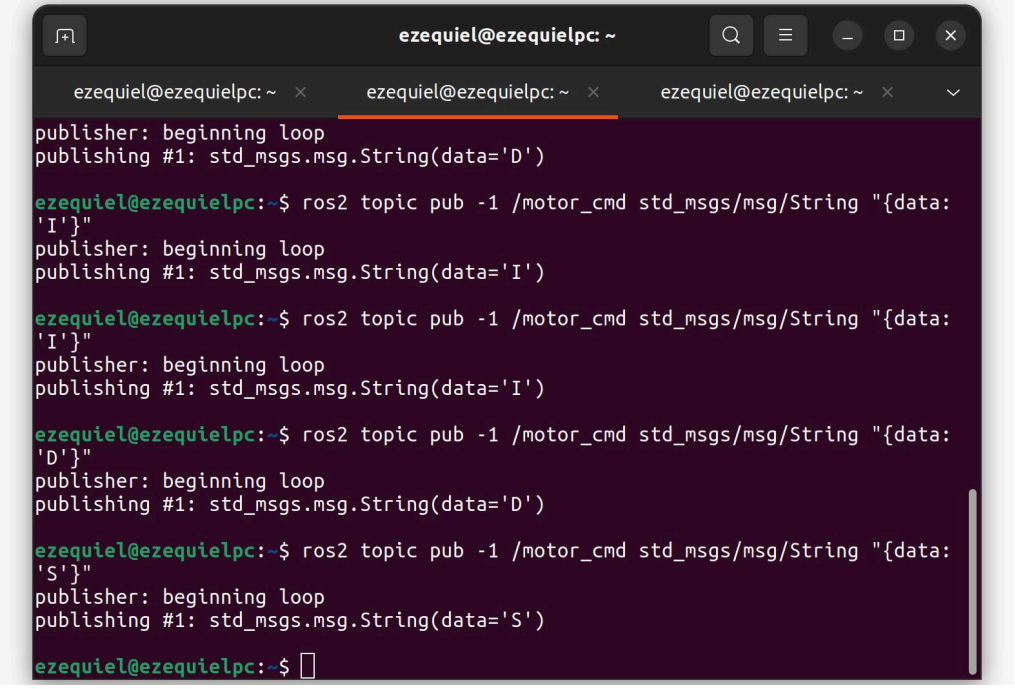


La presencia continua de mensajes recv_message y send_message indica que el nodo embebido permaneció activo y comunicándose sin interrupciones, lo cual demuestra estabilidad en la conexión y correcto funcionamiento del ejecutor.

2. Funcionamiento del Subscriber

Para validar el funcionamiento del suscriptor implementado en el ESP32, se envió manualmente un comando al tópico /motor_cmd utilizando la herramienta de línea de comandos de ROS 2. En la figura mostrada anteriormente se observa la ejecución del comando:

- ros2 topic pub -1 /motor_cmd std_msgs/msg/String "{data:'I'}"



Como se aprecia en la terminal, el sistema indica “publisher: beginning loop” y posteriormente “publishing #1: std_msgs.msg.String(data='I')”, lo que confirma que el mensaje fue publicado correctamente en el tópico correspondiente. Esta evidencia demuestra que la comunicación desde el nodo en ROS 2 hacia el nodo embebido se realizó sin errores.

Posteriormente, se repitió el procedimiento enviando los comandos "D" y "S", correspondientes al giro a la derecha y a la detención del motor, respectivamente. En cada caso, el motor ejecutó la acción esperada de manera inmediata y estable, sin presentar reinicios ni comportamientos anómalos como se muestra en el video de abajo.

Funcionamiento del sistema.mp4

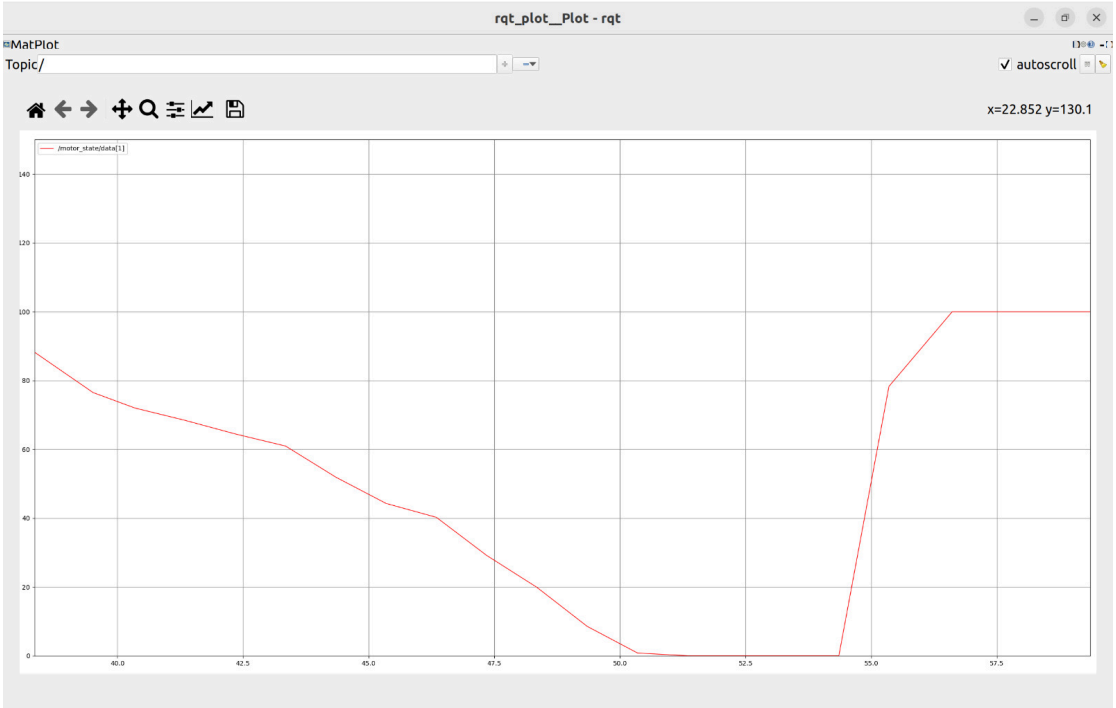
El comportamiento observado confirma que el mensaje fue recibido correctamente por el nodo embebido, que el callback del subscriber se ejecutó conforme a lo programado y que la lógica de control del puente H respondió adecuadamente a cada instrucción. Asimismo, se verificó que la variable interna de estado del motor se actualizó según el comando recibido.

3. Evidencia gráfica mediante rqt_plot

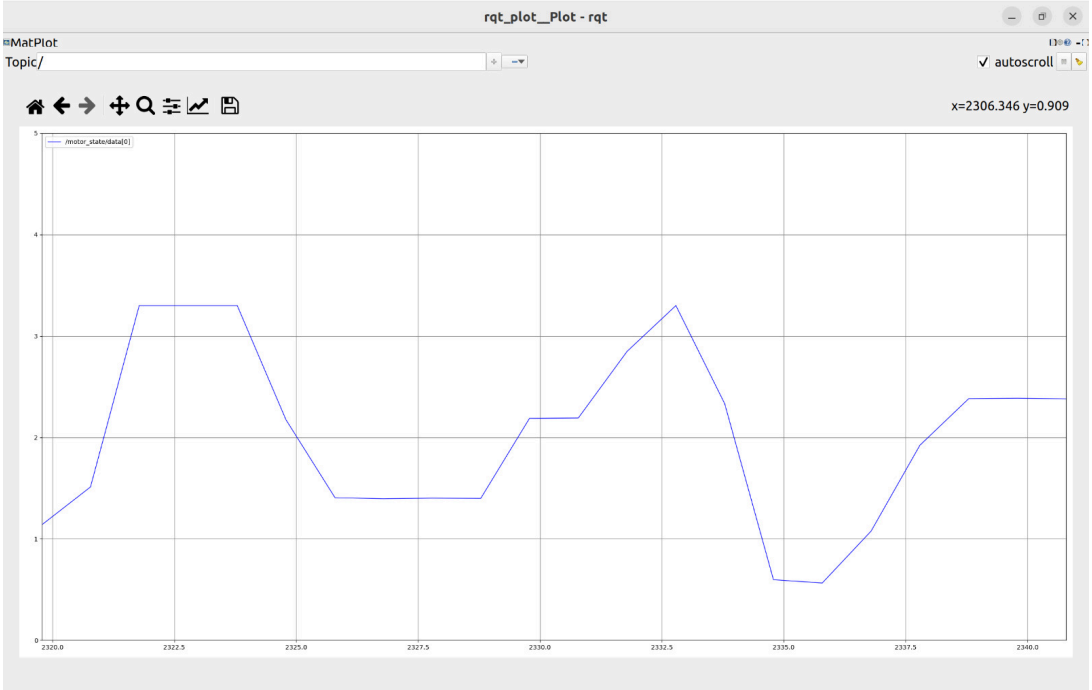
Para analizar el comportamiento dinámico del sistema y verificar la correcta publicación de variables, se utilizó la herramienta rqt_plot, ejecutando el comando:

- ros2 run rqt_plot rqt_plot /motor_state/data[0] /motor_state/data[1]

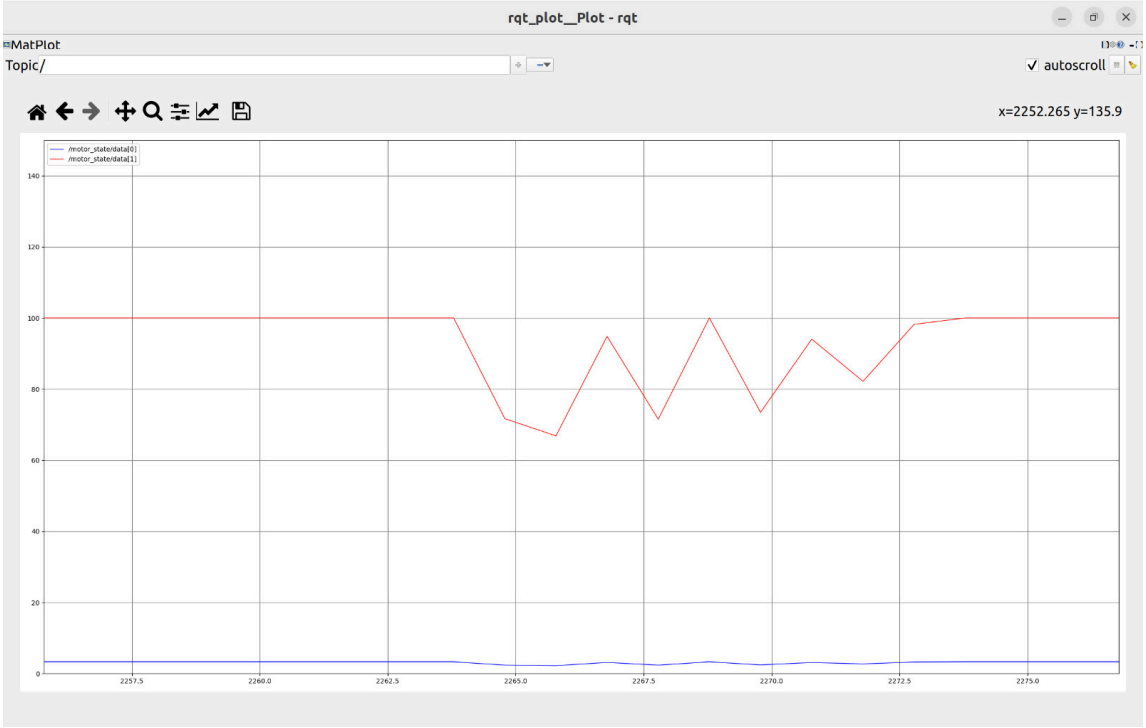
En la primera gráfica presentada se observa el comportamiento del tópico /motor_state/data[1], el cual corresponde al duty cycle del PWM expresado en porcentaje. La señal inicia aproximadamente en un valor cercano al 80 %, posteriormente desciende de manera progresiva hasta alcanzar 0 %, y después incrementa gradualmente hasta llegar a valores cercanos al 100 %. Este comportamiento espaciado en el tiempo coincide con el movimiento manual del potenciómetro, lo que indica que el sistema responde proporcionalmente a la variación de la entrada analógica.



En la segunda gráfica se muestra el comportamiento de /motor_state/data[0], correspondiente al voltaje calculado a partir de la lectura del ADC. En este caso, la señal comienza alrededor de 1.3 V y aumenta progresivamente hasta aproximadamente 3.3 V, que corresponde al máximo voltaje de referencia del ESP32. Posteriormente, el voltaje presenta variaciones acordes con el ajuste del potenciómetro. Esta relación confirma que el escalamiento aplicado al ADC es correcto y que la conversión a voltaje físico se está realizando adecuadamente.



Finalmente, en la tercera gráfica se presentan ambos tópicos simultáneamente. En ella se aprecia claramente que el voltaje y el duty cycle varían de forma proporcional y sincronizada: cuando el voltaje aumenta, el porcentaje de PWM también se incrementa; cuando el voltaje disminuye, el duty cycle se reduce de manera equivalente. Esta correlación directa demuestra que el sistema mantiene una relación lineal entre la señal analógica de entrada y la señal PWM generada, validando el correcto funcionamiento del mapeo implementado.



Conclusiones

El desarrollo de la presente práctica permitió integrar exitosamente un sistema de control de velocidad de motor basado en PWM con una arquitectura distribuida utilizando ROS 2 y micro-ROS en el ESP32. A lo largo del proceso se abordaron tanto aspectos de control electrónico como de comunicación entre nodos, lo que permitió comprender de manera integral la interacción entre hardware y software en sistemas embebidos conectados a un entorno robótico.

Los objetivos planteados fueron alcanzados satisfactoriamente. Se logró implementar un nodo embebido capaz de publicar variables críticas del sistema, tales como el voltaje calculado y el duty cycle, así como recibir comandos externos para el control del motor. La comunicación con el agente micro-ROS se estableció de forma estable, como se evidenció en los registros de la terminal, y los comandos enviados desde ROS 2 fueron recibidos y ejecutados correctamente por el ESP32. Asimismo, mediante el uso de herramientas gráficas como rqt_plot, se validó que las variables publicadas presentaran un comportamiento proporcional y coherente con la entrada analógica del sistema.

En cuanto a posibles mejoras, se podría optimizar la metodología implementando filtrado digital para reducir pequeñas fluctuaciones en la señal analógica. Además, sería ideal estructurar el código en módulos más desacoplados, facilitando su mantenimiento y escalabilidad hacia aplicaciones más complejas, como control en lazo cerrado o integración con sensores adicionales.

En conclusión, la práctica no solo permitió cumplir los objetivos técnicos establecidos, sino que también contribuyó a una comprensión más profunda de los principios de comunicación distribuida, control de motores y procesamiento de señales en sistemas embebidos. La integración exitosa entre hardware y ROS 2 demuestra la viabilidad del enfoque implementado y sienta bases sólidas para desarrollos futuros en aplicaciones robóticas y de automatización.

Referencias Bibliográficas (APA)

- Manchester Robotics. (2024). Mini Challenge 3 (MCR2) [Presentación de PowerPoint].
- Open Robotics. (2026). ROS 2 Humble Hawksbill Documentation. Recuperado de https://docs.ros.org/en/humble/
- micro-ROS. (2026). micro-ROS documentation. https://micro.ros.org/
- Espressif Systems. (2026). ESP-IDF Programming Guide. https://docs.espressif.com/